Informe Tecnico: Agenda de Contactos con Acceso Aleatorio

Estudiante: Josue Soch | Repositorio Git: Lab2_Archivos | Octubre de 2026

1. Estructura del Registro de Tamano Fijo (120 Bytes)

Todos los contactos se almacenan en el archivo 'contactos.dat' en codificacion UTF-8 utilizando registros de longitud estrictamente fija. Cada campo se rellena con espacios a la derecha mediante el metodo .ljust() hasta completar su longitud maxima definida, sumando exactamente 120 bytes por registro:

Campo	Longitud Fija	Rango de Bytes	Descripcion y Relleno
id	6 caracteres	Bytes 0 - 5	Entero unico identificador (.ljust(6))
nombre	30 caracteres	Bytes 6 - 35	Nombre completo del contacto (.ljust(30))
telefono	15 caracteres	Bytes 36 - 50	Numero telefonico (.ljust(15))
correo	40 caracteres	Bytes 51 - 90	Correo electronico (.ljust(40))
ciudad	28 caracteres	Bytes 91 - 118	Ciudad de residencia (.ljust(28))
Salto (\n)	1 caracter	Byte 119	Delimitador de linea fijo (LF = 1 byte)
TOTAL	**120 bytes**	**120 bytes / reg**	**Tamano constante garantizado por registro**

2. Implementacion del Acceso Aleatorio e In-Place con seek() y tell()

Dado que todos los registros miden exactamente RECORD_SIZE = 120 bytes, la posicion exacta (offset en bytes) de cualquier registro N se calcula en tiempo constante O(1) mediante la formula:

target_offset = record_number * 120

Con f.seek(target_offset) el puntero de archivo salta directamente al byte inicial del contacto sin recorrer los registros previos. Al editar un contacto (modificacion in-place), se abre el archivo en modo 'r+' y se sobrescriben unicamente esos 120 bytes. f.tell() indica la posicion actual dentro del archivo antes y despues de cada operacion.

```
# Fragmento de contact_manager.py: Lectura directa y Modificacion In-Place
def read_record_by_number(self, record_number):
    target_offset = record_number * RECORD_SIZE  # Ej: Registro #3 * 120 = byte 360
    with open(self.filename, 'r', encoding='utf-8', newline='\n') as f:
        f.seek(target_offset)           # Salto directo O(1) sin leer el archivo completo
        current_pos = f.tell()          # Posicion inicial antes de leer (ej. 360)
        raw_data = f.read(RECORD_SIZE)  # Lee exactamente 120 bytes
        end_pos = f.tell()              # Posicion final tras lectura (ej. 480)
    return self.parse_record(raw_data), current_pos, end_pos

def update_contact(self, contact_id, nombre, telefono, correo, ciudad):
    record_number = self.index[str(int(contact_id))]
    target_offset = record_number * RECORD_SIZE
    record = self.format_record(contact_id, nombre, telefono, correo, ciudad)
    with open(self.filename, 'r+', encoding='utf-8', newline='\n') as f:
        f.seek(target_offset)           # Se ubica exactamente en el registro a editar
        start_pos = f.tell()
        f.write(record)                 # Sobrescribe SOLO los 120 bytes de ese contacto
        end_pos = f.tell()
    self.save_hash()                    # Actualiza firma SHA-256
    return record_number, start_pos, end_pos
```

3. Funcionamiento del Indice Primario (indice.json) y Eficiencia

El archivo 'indice.json' mantiene un diccionario clave-valor que relaciona el ID unico de cada contacto con su numero de registro fisico dentro de 'contactos.dat' (por ejemplo: {"101": 0, "102": 1, "105": 2, "108": 3}).

¿Por que una busqueda con indice es mas eficiente que una busqueda secuencial?
- Busqueda Secuencial O(N): Requiere abrir el archivo y recorrer registro por registro desde el byte 0 hasta encontrar el ID buscado. Si existen 100,000 contactos, buscar el ultimo requiere 100,000 lecturas de disco.
- Busqueda con Indice Primario + seek() O(1): Consulta primero el ID en el diccionario del indice cargado desde 'indice.json' (busqueda en tabla hash O(1) en memoria) para obtener el numero de registro, e inmediatamente ejecuta f.seek(numero_registro * 120) y lee unicamente 120 bytes. Sin importar cuantos miles de contactos existan, siempre realiza un solo salto directo y una sola lectura en disco.

```
// Contenido real generado en 'indice.json':
{
    "101": 0,
    "102": 1,
    "105": 2,
    "108": 3,
    "112": 4,
    "115": 5,
    "120": 6,
    "125": 7
}
```

4. Evidencia de Verificacion de Integridad SHA-256 (Archivo Alterado)

Cada vez que se guarda o modifica un contacto, se calcula el hash SHA-256 de 'contactos.dat' (modulo hashlib) y se guarda en 'contactos.sha256'. Al iniciar la aplicacion se recalcula y compara; cuando se altero manualmente el Registro #0 fuera de la aplicacion ('Josue' -> 'Xosue'), el sistema detecto la discrepancia de inmediato:



5. Comparacion del Tamano Original contra el Respaldo Comprimido ZIP

La opcion 'Herramientas -> Crear respaldo ZIP' utiliza el modulo 'zipfile' (ZIP_DEFLATED) para comprimir 'contactos.dat' e 'indice.json'. Como los registros de tamano fijo rellenan cada campo con espacios repetitivos mediante .ljust(), la compresion DEFLATE reduce significativamente el tamano en disco:

Archivo / Metrica	Tamano Original	Tamano en ZIP	Ahorro / Reduccion
contactos.dat (8 registros x 120 B)	960 bytes	Incluido en ZIP	Espacios comprimidos
indice.json (8 entradas JSON)	123 bytes	Incluido en ZIP	Estructura JSON
TOTAL COMPARATIVO (respaldo_agenda.z	**1083 bytes**	**642 bytes**	**-441 B (40.72%)**

6. Capturas de Pantalla del Funcionamiento de la Aplicacion

Figura 6.1: Ventana principal con el menu completo, busqueda por ID mediante indice.json, edicion in-place y tabla mostrando el # de Registro y el Offset exacto reportado por tell():

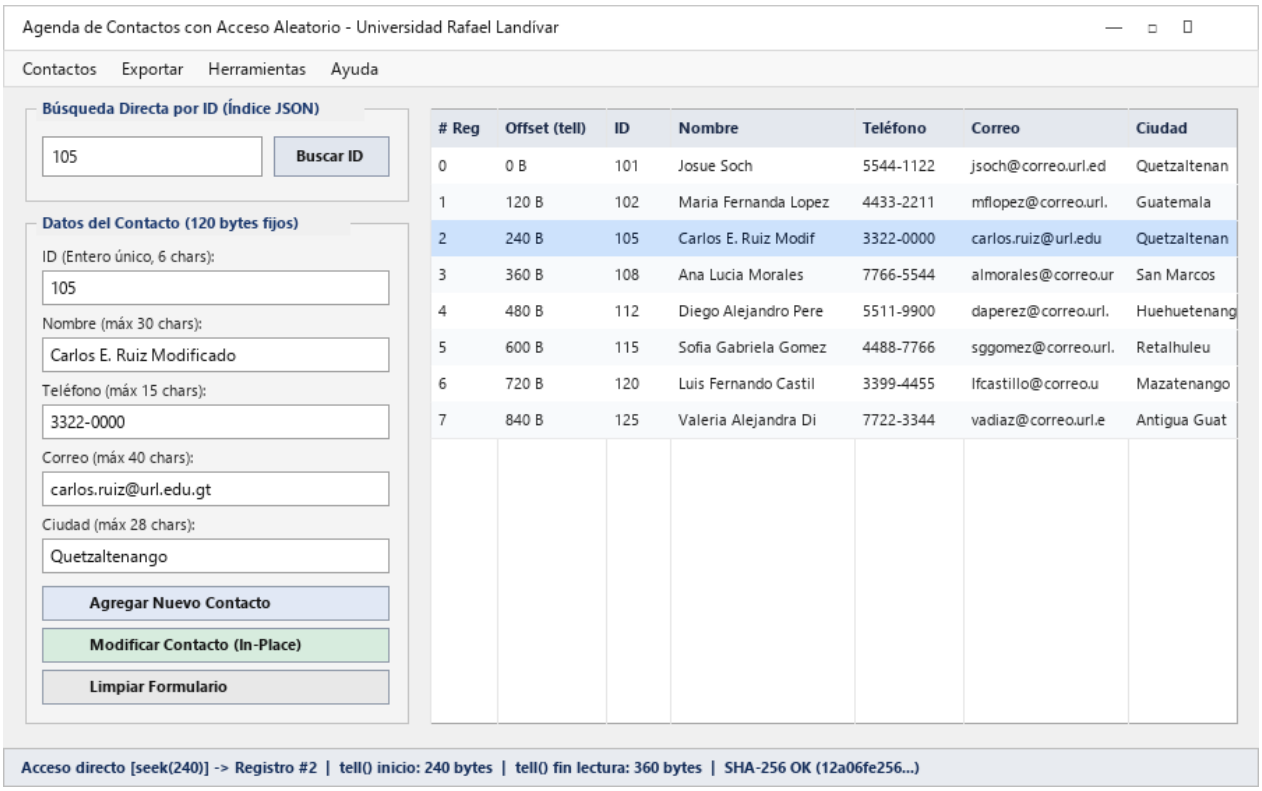
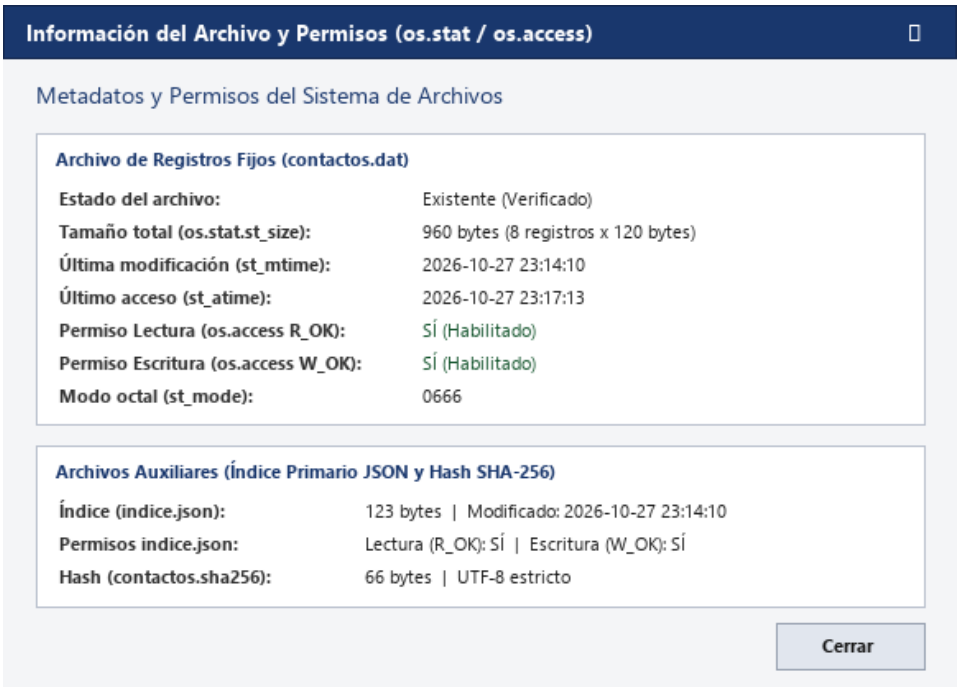


Figura 6.2: Ventana de 'Informacion del archivo' mostrando tamano, fecha de ultima modificacion y permisos de lectura y escritura obtenidos con os.stat() y os.access():


Información del Archivo y Permisos (os.stat / os.access)
Metadatos y Permisos del Sistema de Archivos
Archivo de Registros Fijos (contactos.dat)
Estado del archivo: Existente (Verificado)
Tamaño total (os.stat.st_size): 960 bytes (8 registros x 120 bytes)
Última modificación (st_mtime): 2026-10-27 23:14:10
Último acceso (st_atime): 2026-10-27 23:17:13
Permiso Lectura (os.access R_OK): SÍ (Habilitado)
Permiso Escritura (os.access W_OK): SÍ (Habilitado)
Modo octal (st_mode): 0666
Archivos Auxiliares (Índice Primario JSON y Hash SHA-256)
Índice (indice.json): 123 bytes | Modificado: 2026-10-27 23:14:10
Permisos indice.json: Lectura (R_OK): SÍ | Escritura (W_OK): SÍ
Hash (contactos.sha256): 66 bytes | UTF-8 estricto
Cerrar